Operational Forensic Audit and Strategic Roadmap for Phase 4: The Stela Autopoietic Ecosystem and the Library Trinity Deep Dive

1. Executive Strategic Analysis: The Architectural Inflection of Phase 3

The contemporary software engineering landscape currently undergoes a paradigm shift of unprecedented magnitude, transitioning from static, human-maintained repositories to dynamic, agentic ecosystems.[1] The Stela platform, as evidenced by the forensic artifacts provided in the repository dump and the associated "AI Agent Constitution Review," stands at the vanguard of this transition.[4] This report provides an exhaustive operational forensic audit of the platform's current state, specifically validating the execution of Dispatch Packet DP-OPS-0026, and analyzes the systemic implications of the newly established Policy of Truth (PoT) framework.

The analysis proceeds from the fundamental understanding that Stela serves not merely as a codebase but as an implementation of autopoietic systems theory within a software development lifecycle.[4] Autopoiesis, a concept originally derived from biological cybernetics by Maturana and Varela, describes systems capable of reproducing and maintaining themselves within a topological boundary. In the context of Stela, this translates to a "Project Factory" architecture where the system's operating kernel (ops/) stands decoupled from its variable payloads (projects/), creating a clean separation of concerns analogous to the "kernel-user space" division in monolithic operating systems.[4] This distinction remains critical because, unlike allopoietic systems which produce something other than themselves (like a car assembly line producing cars), an autopoietic system's primary product serves as its own continued organization.

1.1 The Theoretical Framework of Autopoietic Software

To rigorously evaluate the success of the recent Phase 3 transition (DP-OPS-0026), one must first deconstruct the theoretical underpinnings that govern Stela's architecture. Stela represents a stark departure from traditional "Citizen Models" of AI interaction toward a "Mechanistic Doctrine".[4] This transition functions not merely as a change in tooling but as a fundamental reorientation of the "Social Contract" between the human operator and the artificial agent.

1.1.1 The Mechanistic Doctrine vs. The Citizen Model

The "Citizen Model" of AI interaction anthropomorphizes artificial agents, treating them as trusted collaborators capable of discerning intent, understanding nuance, and "doing the right thing" amidst ambiguity.[8] This model relies heavily on social contracts, implied context, and high-trust environments. While effective in human-centric teams where shared cultural context bridges gaps in documentation, the Citizen Model introduces catastrophic risks in agentic systems due to "Epistemic Drift".[4] Epistemic Drift occurs when a corruption in the rule-defining mechanism—such as a template, a linter, or a prompt instruction—propagates exponentially through the system, creating a cascade of architectural divergence.

In contrast, the "Mechanistic Doctrine," which Stela formally adopted in Phase 3, posits that the system must physically and deterministically prevent the agent from performing unauthorized actions.[4] It rejects trust in favor of verification. Under this doctrine, an agent does not receive a request to follow a style guide; the system undergoes engineering to crash if the style guide suffers violation. This shift proves critical because, unlike human operators who may fatigue, rationalize shortcuts, or "bend the rules" for expediency, autonomous agents operate with relentless speed and absolute literalism. If a vulnerability exists in the "Project Factory" (specifically the ops/bin/project binary), every project instantiated by that factory inherits the vulnerability, leading to systemic fragility.[4]

The successful execution of DP-OPS-0026 enshrined this doctrine within the Preamble of the PoT.md file, explicitly stating: "The system prevents action that violates policy. It does not request compliance".[4] This constitutes a historic milestone for the repository, moving it from a state of "aspirational governance" to "deterministic enforcement."

1.1.2 From System Injection to Project Election

Phase 3 stands operationally defined by the architectural pivot from "System Injection" to "Project Election".[4] In earlier iterations of the Stela platform, the system likely injected a global context into every agent session. While this ensured that agents held access to all available tools, it led to "Context Haze"—a degradation of AI reasoning capabilities caused by the saturation of irrelevant information.[1] This phenomenon mirrors the "information overload" described in knowledge management theories, where the absence of a Single Source of Truth (SSOT) leads to cognitive paralysis.

The new autopoietic model mandates that the global system (ops/) remains sterile, containing only the absolute minimum tools required for construction. The capabilities—Agents, Tasks, and Skills—reside in the "Library Trinity" (docs/library/) and load into the runtime environment only when explicitly "elected" by a project's STELA.md configuration file.[4] This reduces token overhead and sharpens agent focus, but it introduces a new dependency: Referential Integrity. If a project elects a capability that does not exist (a "Dead Pointer"), the autopoietic process fails. Thus, the verification of DP-OPS-0026 centers on whether the system successfully implemented the linters and guards necessary to enforce this integrity, ensuring

that the "machine specification" remains valid at all times.[4]

2. Forensic Verification of DP-OPS-0026 Execution

This section presents a forensic audit of the repository state (HEAD 7bc5050de) to determine if the execution of DP-OPS-0026 met its strategic objectives. The analysis relies on the "State of Play" (SoP.md) ledger, the "Policy of Truth" (PoT.md), and the file manifest provided in the repository dump.[4]

2.1 The Evidence of Execution: Analyzing the Ledger

The primary artifact for verification serves as SoP.md (State of Play). In the Stela governance model, SoP.md functions as the immutable history ledger; a task legally achieves "complete" status only when recorded here. This mirrors the function of a blockchain or a secure audit log in financial systems, where the record of the transaction acts as the transaction itself.[4]

Audit Finding: The SoP.md file contains a specific entry for 2026-02-05 - DP-OPS-0026: Phase 3 Semantic Refactoring & Structural Hygiene.[4] This entry explicitly logs the following actions:

- **Refactoring of PoT.md nomenclature:** The log confirms the transition to "Governance Engine," "Core," "Federation," and "Tiered Jurisdiction."
- **Structural Refactoring:** The log records the replacement of docs/ops/INDEX.md with an architectural docs/ops/README.md and the removal of the redundant docs/security/ surface.
- **Enforcement Hardening:** The log notes updates to tools/lint/truth.sh and tools/lint/style.sh to enforce the new terminology and markdown robustness.

Conclusion: The presence of this detailed entry in the canonical ledger provides *prima facie* evidence that the governance requirements for recording the work met satisfaction. The system acknowledges the change, satisfying the "Equilibrium Maintenance" directive of the PoT. This adherence to the ledger prevents the "hidden drift" often seen in systems where code changes occur without corresponding governance updates.[4]

2.2 Deep Dive: The Monolithic Constitution (PoT.md)

The creation and refinement of PoT.md stands as the centerpiece of Phase 3. A structural analysis of the file confirms that it effectively merges the disparate governance logic into a coherent whole, eliminating the "Scattered Truth" problem identified in the Constitution Review.[4]

2.2.1 Semantic Refactoring: The Retirement of the Sheriff

The most significant aspect of DP-OPS-0026 involved the semantic refactoring of governance terms. The forensic audit of PoT.md (HEAD 7bc5050de) confirms the successful

excision of the legacy "Wild West" metaphors in favor of precise, systems-theory terminology.[4]

Legacy Term	New Term	Location Verified	Implication
Sheriff	**Governance Engine**	PoT.md Sec 2	Moves from a persona-based enforcement model (discretionary) to a machine-based model (deterministic). The Governance Engine is defined as "Integrator plus Automation".[4]
Colonies	**Federation**	PoT.md Sec 3.2	"Colonies" implied subservience. "Federation" implies semi-autonomy within a constitutional framework, aligning with the "Tiered Jurisdiction" model where projects have local rights.[4]
Crown	**Core**	PoT.md Sec 3.1	"Crown" implied imperial rule. "Core" implies a central kernel or operating system, aligning with the technical reality of the ops/ directory.[4]
Multi-verse	**Tiered**	PoT.md Sec 3	"Multi-verse" implied

	Jurisdiction		disconnected realities. "Tiered Jurisdiction" precisely describes the relationship between the Platform Core and the Project Federation.

This semantic shift serves as more than cosmetic; it functions as *ontological*. By renaming "The Sheriff" to "The Governance Engine," the system explicitly depersonalizes enforcement. A Sheriff might grant a warning; an Engine simply executes a halt command. This aligns perfectly with the Mechanistic Doctrine's assertion that the system "prevents action" rather than "requests compliance".[4]

2.3 Structural Hygiene and the Filing Doctrine

The forensic audit confirms that the physical structure of the repository underwent alteration to match the "Filing Doctrine" (ops/ = Run, docs/ = Explain).[4]

2.3.1 Elimination of docs/security/

The file list in the dump [4] no longer contains docs/security/README.md or the docs/security/ directory. This confirms the successful pruning of redundant pointers. Security policy now stands consolidated within PoT.md Section 5.4, reducing the "Context Haze" caused by fragmented documentation. In information theory, a "pointer to a pointer" (like the old docs/security/README.md) represents inefficient encoding. By forcing security into the central Constitution, Stela ensures that *every* agent loads the security policy as part of the "Small Bundle" context.[4]

2.3.2 Refactoring of docs/ops/

The file docs/ops/INDEX.md, which previously acted as a weak pointer list, underwent replacement by docs/ops/README.md.[4] The content of this new file strictly adheres to the architectural mandate: "The Ops Kernel is the executable companion to docs. Documentation explains and justifies; ops runs." This clearly delineates the separation of concerns, ensuring that human operators look to docs/ for understanding while the system looks to ops/ for execution.

2.4 Linter Hardening: The Mechanistic Guarantee

Governance remains only as real as its enforcement. The audit of tools/lint/truth.sh reveals that the semantic shift receives backing by code, not just policy.

The forbidden_spellings array in tools/lint/truth.sh now explicitly includes: "Sheriff", "Colonies", and "Crown".[4] This constitutes the "Mechanistic Doctrine" in action. The system does not merely *ask* contributors to use the new terms; it *physically prevents* the re-introduction of the old metaphors. If an agent or operator attempts to commit a file referencing "The Sheriff," the linter triggers a hard failure, halting the PR pipeline. This satisfies the PoT preamble: "The system prevents action that violates policy. It does not request compliance".[4]

3. The Library Trinity Deep Dive: Validating the Need for Phase 4

While the Core (ops/, PoT.md, SoP.md) has achieved a state of hardened, autopoietic stability ("Phase 3"), the analysis reveals a critical maturity gap in the **Library Trinity** (docs/library/agents, docs/library/tasks, docs/library/skills). The user's query identifying the need for a "deep dive" into these directories stands strategically sound and urgent. The current state of the agents, tasks, and skills represents a "capability lag" relative to the platform's "governance advance."

3.1 The "Tethering" Gap: Untethered Agents in a Federated World

The forensic audit of the docs/library/agents/ directory (containing R-AGENT-01 through R-AGENT-06) reveals a critical flaw in the current agent architecture: **Lack of Cognitive Tethering**.[4]

Currently, the agents exist defined as generic personas (architect, code-reviewer, integrator). While ops/bin/project injects the project context at runtime, the *internal self-conception* of the agent—its "system prompt personality"—lacks explicit engineering to accept and prioritize that context.

In an agentic workflow, an agent that "thinks" it functions as a generic software architect will default to general best practices (e.g., "Use standard clean code") derived from its training data. However, a Stela Federation Agent must "believe" it functions as a specialized worker operating solely within the bounds of a specific project, governed by that project's STELA.md.[14] If STELA.md mandates a specific, non-standard architectural pattern (e.g., "Use vertical slice architecture over clean architecture"), the agent must adhere to that local law without hesitation.

The Diagnostic: The current agents rely on the *Integrator* (the runtime orchestration) to provide context. They lack the "Recursive Context Check" instruction within their own definition files. They do not explicitly know to look for STELA.md or to treat PoT.md as their binding constraints.[17]

3.2 The Integration Gap: Task-Agent-Skill Disconnect

The user correctly notes that "THE THREE CATEGORIES NEED TO WORK TOGETHER IN COMBINATIONS." Currently, the library components exist somewhat in isolation.

- **Agents** define *who* does the work.
- **Tasks** define *what* sequence of steps constitutes the work.
- **Skills** define *how* specific technical operations are performed.

A "Deep Dive" remains required to weave these into a cohesive "Instructional Graph."

- **Task-Agent Binding:** B-TASK-03 (Integrate Command) [4] begins this by chaining integrator -> code-reviewer -> security-reviewer. However, the forensic audit does not show evidence that the *Agents themselves* possess awareness of this chain. Does the Integrator agent know it holds responsibility for handing off to the Code Reviewer? Or does that logic reside solely in the task file? In a truly autopoietic system, the agents should possess "Team Awareness".[7]
- **Skill-Agent Binding:** Does R-AGENT-06 (Security Reviewer) explicitly know to invoke S-LEARN-05 (Security Protocols)? Or does it rely on generic security knowledge? The "Mechanistic Doctrine" demands that we do not trust the LLM's training data when we possess a verified Skill in the library. The agent definitions should explicitly reference the Skills they carry the expectation to wield.

3.3 The Isolation Requirement: "Their Own World"

The prompt specifies: "THOSE AGENTS NEED TO BELIEVE THEY OPERATE IN THEIR OWN WORLD (projects/*)." This constitutes the implementation of the "Federation" model.[4]

- **Cognitive Containment:** When an agent undergoes instantiation for projects/alpha, it should possess no awareness of or influence by projects/beta.
- **Pathing Discipline:** Agents must undergo rigorous training (via their system prompts in docs/library/agents/) to treat ./ as the project root, not the repo root, whenever possible, or to handle the path translation flawlessly. The ops/bin/project script handles the directory switching [4], but the agent's internal reasoning must align with this relative pathing to prevent "Path Hallucination" (e.g., trying to write to ../../ops/ from inside a project).

4. Operational Analysis of the Library Trinity

To prepare for the deep dive, we analyze the current state of the three library categories based on the visible artifacts and the established patterns in the repository dump.

4.1 Agents (docs/library/agents/)

Current State:

- R-AGENT-01 (Architect) [4]: Defined as a "Senior software architect." Lists tools (Read, Grep, Glob).
- R-AGENT-02 (Code Reviewer) [4]: Defined as a "Senior code reviewer." Mandates strict checks.
- R-AGENT-06 (Security Reviewer) [4]: Focuses on OWASP Top 10.

Deficiencies:

- **Generic Persona:** The descriptions read like job postings for human hires ("You are a senior..."). They lack the "Mechanistic" rigidity of a machine instruction.
- **Missing Jurisdictional Preamble:** No explicit statement defines their legal standing within the Federation. They do not know they are "Federation Agents."
- **Contraction Risk:** While PoT.md bans contractions, the agent prompts themselves must undergo auditing. The user prompt requests a check for contractions like "don't," "can't," "won't" in these files.

4.2 Tasks (docs/library/tasks/)

Current State:

- B-TASK-03 (Integrate Command) [4]: Defines the workflow integrator -> code-reviewer -> security-reviewer.
- B-TASK-01 (Code Review) [4]: Defines the review checklist.

Deficiencies:

- **Implicit Handoffs:** The tasks describe the *flow* but do not define the *interface* between agents. How does the Architect pass context to the Coder? A standardized "Handoff Artifact" (defined in the Task) remains missing.
- **Legacy Terminology:** Tasks created prior to DP-OPS-0026 may still reference "Sheriff" or "Colonies" in their instructional text. A grep audit remains necessary.

4.3 Skills (docs/library/skills/)

Current State:

- S-LEARN-08 (Prune SoP) [4]: Contains a robust ## Provenance block. This constitutes the gold standard.
- S-LEARN-01 through 05: These legacy skills underwent refactoring in DP-OPS-0013/0014 [4], but must undergo re-verification against the Phase 3 "Context Hazard" rules.

Deficiencies:

- **Context Hazard Risk:** Skills serve as "Just-In-Time" (JIT) resources. If they accidentally enter ops/lib/manifests/CONTEXT.md, they pollute the global context.
- **Static Stacks:** S-LEARN-02 defines the "Stela Stack" (Next.js/FastAPI). If a project in the

Federation uses a different stack (e.g., PHP/Laravel), this Skill becomes a liability. Skills must become "Dynamic" or "Polymorphic," capable of adapting to the STELA.md definition of the project.[20]

5. Strategic Roadmap: Phase 4 Implementation Plan (DP-OPS-0027)

The recommendation proves affirmative: **Yes, the time has arrived for a deep dive.** The platform stands structurally sound (Phase 3 complete), but the workforce (Library) requires alignment. This must occur "WITHOUT DRIFT," meaning no new files outside the library should undergo creation, and PoT.md invariants must undergo respect.

Proposed Dispatch Packet: DP-OPS-0027 (Phase 4 Library Hardening)

5.1 Objective

Standardize the Library Trinity (agents/, tasks/, skills/) to enforce Tiered Jurisdiction, tether all Federation logic to STELA.md, and ensure inter-category operability under the Mechanistic Doctrine.

5.2 Scope of Work

A. Agent Refactoring (docs/library/agents/*)

For every agent file (R-AGENT-01 through 06):

1. **Inject Jurisdiction Preamble:** Add a standard block defining the agent's legal standing.
    - *Draft Text:* "JURISDICTION: Federation. AUTHORITY: Derived from STELA.md. CONSTRAINT: Cannot alter Core."
2. **Tethering Instruction:** Explicitly command the agent to read and cite STELA.md before taking action.
    - *Draft Text:* "PRIMARY DIRECTIVE: Align all output with the local project constitution (STELA.md). If STELA.md is silent, default to Platform Core standards."
3. **Contraction Sweep:** Remove any contractions in the prompt text (e.g., change "You're" to "You are", "Don't" to "Do not").

B. Task Composition (docs/library/tasks/*)

For every task file (B-TASK-01 through 10):

1. **Agent Binding:** Explicitly list which R-AGENT IDs hold authorization to execute the task.
2. **Skill Binding:** Explicitly list which S-LEARN IDs should undergo loading or referencing during execution.
3. **Workflow Hygiene:** Ensure tasks do not encourage "Drift" (e.g., creating files outside of the project root).

C. Skill Verification (docs/library/skills/*)

1. **Context Hazard Check:** Verify no skill accidentally entered ops/lib/manifests/CONTEXT.md (re-run tools/lint/context.sh).
2. **Provenance Audit:** Ensure every skill possesses a valid ## Provenance block.
3. **Stack Agnosticism:** Review S-LEARN-02 and others for hard-coded stack assumptions. If found, wrap them in conditional logic: "IF STELA.md declares Stack: Next.js, THEN apply this skill."

5.3 Execution Logic

This work must undergo iterative performance.

1. **Draft:** Create the updated content locally.
2. **Verify:** Run tools/lint/truth.sh to ensure no "Sheriff/Colonies" slip in.
3. **Simulate:** Use ops/bin/project work <test-id> --agent=architect to verify the agent "wakes up" with the correct context and tethering instructions.

6. The Federation Protocol: A New Interaction Model

To satisfy the user's request "WITHOUT DRIFT," we must execute a surgical operation on the Library. We define the **Federation Protocol** as the set of rules governing agent behavior within the projects/ scope.

6.1 Principles of the Federation Protocol

1. **Local Supremacy:** STELA.md > Global Best Practices. The agent must respect the local law even if it contradicts its training data (e.g., "Use Tabs" vs "Use Spaces").
2. **Core Obedience:** PoT.md (Global) > STELA.md (Local) > Agent Training. The agent must never violate the Constitution (e.g., "No Secrets in Code") even if STELA.md requests it.
3. **Isolation:** Actions in Project A must never affect Project B. Agents must operate with "Cognitive Containment," treating the project directory as their entire universe.

6.2 Implementation in ops/bin/project

While the binary itself functions as part of the Core (and thus lies outside the Library Deep Dive scope), the *usage* of the binary by Agents in Tasks must reflect this protocol. The Tasks must instruct agents to use ops/bin/project commands to interact with the system, rather than raw shell commands, ensuring that the "Governance Engine" stays in the loop.[4]

7. Conclusion

The Stela ecosystem has achieved a "Professional Autopoiesis" state. The Core stands solid, the Governance Engine operates actively, and the history remains secure. The implementation of DP-OPS-0026 proved a complete success, creating the necessary constitutional

foundation for autonomous scaling.

The "Deep Dive" into the Library serves not just as a maintenance task; it constitutes the **Cognitive Awakening** of the system. By tethering agents to their specific STELA.md realities, we move from "generic AI assistance" to "specialized, constitutionally-bound federated labor." This alignment represents the final step in ensuring that when the system replicates, it replicates order, not entropy.

Recommendation: Proceed immediately with **DP-OPS-0027**. The system stands ready.

Appendix: Data Tables

Table 1: Terminology Migration Status (DP-OPS-0026 Audit)

Legacy Term	New Term	Location Verified	Status
Sheriff	Governance Engine	PoT.md Sec 2	**Complete**
Colonies	Federation	PoT.md Sec 3.2	**Complete**
Crown	Core	PoT.md Sec 3.1	**Complete**
Multi-verse	Tiered Jurisdiction	PoT.md Sec 3	**Complete**

Table 2: File Structure Refactoring Status

Artifact	Previous Location	New Location	Purpose
Ops Manual	ops/INDEX.md	docs/ops/README.md	Align with Filing Doctrine (Explain vs Run)
Security Docs	docs/security/	PoT.md Sec 5.4	Consolidate Truth / Reduce Context Haze
Legacy Truth	TRUTH.md	storage/archives/	Tombstoning Legacy Canon (Anti-Drift)

Table 3: Library Trinity Status & Phase 4 Needs

Component	Current Status	Phase 4 Refactoring Need
Agents	Generic Personas (R-AGENT-01)	**Tethering:** Bind to STELA.md & PoT.md explicitly. Remove contractions.
Tasks	Loose Workflows (B-TASK-03)	**Composition:** Explicitly chain Agents + Skills. Define "Handoff Artifacts."
Skills	Static Procedures (S-LEARN-02)	**Dynamic Scope:** Adapt to Project Registry types. Ensure Provenance blocks.

Detailed Analysis: Stela Autopoietic Ecosystem & Phase 4 Roadmap

1. Introduction: The Autopoietic Turn

The Stela repository has undergone a fundamental transformation, moving from a standard software project to an **Autopoietic System**—a system capable of self-production and self-maintenance.[5] This transition, marked by the shift from "Phase 2" (Compression) to "Phase 3" (Autopoiesis), receives definition through the adoption of the **Mechanistic Doctrine**. Unlike traditional "Human-in-the-Loop" models where AI suggests and humans decide, the Mechanistic Doctrine posits that the system must deterministically prevent policy violations through code constraints.[9]

The recent execution of **DP-OPS-0026** constituted the capstone of this transition. It harmonized the repository's language with its architecture, replacing the "Wild West" metaphors of "Sheriff" and "Colonies" with the precise, federalist terminology of "Governance Engine" and "Federation." This report validates that success and outlines the necessary next steps to propagate this doctrine into the system's operational workforce: the AI Agents.

2. Forensic Audit of DP-OPS-0026

The validity of the "Phase 3" transition rests on the verifiable execution of DP-OPS-0026. Using the provided repository dump [4], we confirm the following state changes.

2.1 Constitutional Refactoring (PoT.md)

The **Policy of Truth (PoT.md)** serves as the "Supreme Statute" of the repository. A structural analysis confirms it has undergone rewriting to reflect the new nomenclature.

2.1.1 The Governance Engine (Enforcement)

Section 2 of PoT.md now bears the title **"The Governance Engine (Enforcement)"**. It defines the Engine as "the Integrator plus Automation (linters, repo-gates, and binaries)".[4] This definition proves crucial because it unifies the AI (Integrator) and the Code (Automation) into a single legal entity. The mandate "Stop work. Reject PRs. Kill processes" provides a binary instruction set compatible with the **Mechanistic Doctrine**.

2.1.2 The Tiered Jurisdiction (Core vs. Federation)

Section 3 of PoT.md establishes the **Tiered Jurisdiction** model.[4]

- **The Core (Platform):** Defined as ops/, docs/, tools/, and .github/. Law: "Absolute PoT enforcement."
- **The Federation (Projects):** Defined as projects/. Law: "Structural adherence... Local law applies to project content."

This distinction solves the "One Size Fits All" problem. By allowing the "Federation" to operate under "Local Law" (defined in STELA.md), the system enables innovation at the edge while maintaining stability at the core. This mirrors the biological concept of "Cell Differentiation" in autopoietic systems.[5]

2.2 Structural Hygiene (The Filing Doctrine)

The **Filing Doctrine** (Section 1.1 of PoT.md) mandates a strict separation of concerns: ops/ for execution, docs/ for explanation.

- **docs/ops/README.md**: This file now exists [4], replacing the previous ops/README.md. This change forces a "physical" separation between the binary tools and their documentation. An agent scanning ops/ will now see only code, reducing token overhead and cognitive load ("Context Haze").[1]
- **docs/security/**: This directory underwent successful purging. Security policy now stands consolidated in **PoT.md Section 5.4**.[4] This consolidation functions as a critical "Anti-Drift" measure. In dispersed systems, security policies often rot in forgotten README files. By forcing security into the central Constitution, Stela ensures that *every*

agent loads the security policy as part of the "Small Bundle" context.[4]

2.3 Verification of Enforcement Tools

The dump of tools/lint/truth.sh reveals the code-level enforcement of the new semantic standard.

Bash

```
forbidden_spellings=(
  "Steela"
  "Stila"
  "Stella"
  "Sheriff"
  "Colonies"
  "Crown"
)
```

Source: tools/lint/truth.sh [4]

Insight: This script functions as the "immune system" of the autopoietic organism. By treating "Sheriff" as a spelling error comparable to "Steela" (a typo), the system degrades the legacy metaphor to the status of noise. This effectively "unlearns" the old culture, a notoriously difficult task in human organizations but a trivial one in code-governed autopoietic systems.[6]

3. The Library Trinity: Agents, Tasks, Skills

With the Core hardened, attention must shift to the **Library Trinity** (docs/library/). This directory contains the "genetic material" for the system's workforce. The user's query regarding a "Deep Dive" into these files finds validation in the current repository state.

3.1 The "Tethering" Problem

The current agent definitions (e.g., R-AGENT-01) appear generic. While they mention STELA.md [4], they do not forcefully center it as the *primary* constraint.

The Risk of Generic Intelligence:

Large Language Models (LLMs) undergo training on the "average" of the internet. Without specific "Tethering," an agent defaults to "average" best practices.

- *Scenario:* A user creates a project projects/legacy-php with a STELA.md that mandates

"No namespaces, use include."
- *Untethered Agent:* Ignores STELA.md, sees .php files, and attempts to refactor into Modern PHP with namespaces (because the LLM knows that constitutes "better" practice).
- *Tethered Agent:* Reads STELA.md, accepts the "No namespaces" constraint as absolute law, and writes compliant legacy code.

The Solution: Recursive Context Injection

The "Deep Dive" must rewrite the Agent Prompts (R-AGENT-*.md) to implement a **Recursive Context Check**.

- *Instruction:* "Upon initialization, you must locate and read STELA.md. You must explicitly acknowledge the constraints within. If STELA.md contradicts your general training, STELA.md wins.".[17]

3.2 Interoperability and Combinations

The user notes: "THE THREE CATEGORIES NEED TO WORK TOGETHER IN COMBINATIONS." Current state analysis of B-TASK-03 (Integrate Command) shows a linear chain: integrator -> code-reviewer -> security-reviewer.[4]

Gap Analysis:

- Does R-AGENT-04 (Integrator) know *how* to hand off to R-AGENT-02 (Code Reviewer)?
- Currently, this orchestration seems to happen in the ops/bin/project binary or the B-TASK file instructions.
- **Recommendation:** Enhance the Agent definitions to be "Team Aware." The Integrator's prompt should include: "You are the Architect of the Federation. Your output will be scrutinized by the Code Reviewer. Optimize for clarity to ensure their pass."
- This creates a **Multi-Agent System (MAS)** where agents anticipate the needs of their peers, reducing friction in the handover.[7]

3.3 Skill Harvesting and Provenance

The dump shows docs/library/skills/S-LEARN-08 [4] includes a robust provenance block:

Provenance

- **Captured:** 2026-02-04 16:36:53 UTC
- **DP-ID:** Not provided
- **Branch:** work/phase-two-doctrine-2026-02-04
  ...
  This serves as an excellent model. It fulfills the "Autopoietic" requirement that the system remembers *how* it learned.
  Refactoring Goal: Ensure *all* skills (S-LEARN-01 through 07) attain this standard. Older

skills might lack this forensic detail. The Deep Dive must "backfill" this metadata where possible or explicitly mark it as "Legacy/Foundational."

4. Phase 4 Strategy: The Federation Protocol

To satisfy the user's request "WITHOUT DRIFT," we must execute a surgical operation on the Library.

4.1 The Federation Protocol (Definition)

We define the **Federation Protocol** as the set of rules governing agent behavior within the projects/ scope.

1. **Local Supremacy:** STELA.md > Global Best Practices.
2. **Core Obedience:** PoT.md (Global) > STELA.md (Local) > Agent Training.
3. **Isolation:** Actions in Project A must never affect Project B.

4.2 Implementation Steps (DP-OPS-0027)

1. **Refactor Agents (docs/library/agents/):**
   - Rewrite R-AGENT-01 through 06.
   - **Action:** Add a "Jurisdiction Block" to the top of each system prompt.
   - **Action:** Remove all contractions.
   - **Action:** Add "Tethering Triggers" (e.g., "Confirm STELA.md loaded").
2. **Refactor Tasks (docs/library/tasks/):**
   - Rewrite B-TASK-01 through 10.
   - **Action:** Update terminology (Federation/Core).
   - **Action:** Explicitly link Tasks to the Skills required to execute them (e.g., B-TASK-01 *requires* S-LEARN-04 Coding Standards).
3. **Refactor Skills (docs/library/skills/):**
   - **Action:** Audit all skills for "Context Hazards." Ensure they exist strictly as JIT (Just-In-Time) resources and do not load globally.
   - **Action:** Ensure S-LEARN-02 (Project Guidelines) functions dynamically enough to handle different project types defined in the Project Registry.[4]

5. Conclusion

The implementation of **DP-OPS-0026** proved successful. The Stela ecosystem now possesses a hardened, professional Constitution (PoT.md) and a rigorous Enforcement Engine (truth.sh). The "Sheriff" has retired; the "Governance Engine" is online.

However, a Constitution remains useless without citizens who obey it. The current Agents function well but remain legally vague. They require deputization into the Federation. The requested "Deep Dive" into docs/library/* constitutes the correct and necessary next step. It will transform the library from a folder of markdown files into a **Cognitive Operating System**

where every agent knows its place, its power, and its limits.

Status: DP-OPS-0026 Complete.

Next Action: Initialize DP-OPS-0027 for Library Deep Dive and Federation Protocol implementation.

Works cited

1. What Are Agentic Workflows? A 2025 Guide to Everything You Need To Know - Vonage, accessed February 5, 2026, https://www.vonage.com/resources/articles/agentic-workflows/
2. The Agentic Software Development Life Cycle (A-SDLC) - Architecture & Governance Magazine, accessed February 5, 2026, https://www.architectureandgovernance.com/applications-technology/the-agentic-software-development-life-cycle-a-sdlc/
3. AI Agents: Evolution, Architecture, and Real-World Applications - arXiv, accessed February 5, 2026, https://arxiv.org/html/2503.12687v1
4. Stela AI Constitution Dispatch Packet Generation.pdf
5. A New Class of Autopoietic and Cognitive Machines - MDPI, accessed February 5, 2026, https://www.mdpi.com/2078-2489/13/1/24
6. Supply chain as an autopoietic learning system - Emerald Publishing, accessed February 5, 2026, https://www.emerald.com/scm/article/19/1/10/354386/Supply-chain-as-an-autopoietic-learning-system
7. Designing Multi-Agent Intelligence - Microsoft for Developers, accessed February 5, 2026, https://developer.microsoft.com/blog/designing-multi-agent-intelligence
8. Rise of Machine Agency: A Framework for Studying the Psychology of Human–AI Interaction (HAII) | Journal of Computer-Mediated Communication | Oxford Academic, accessed February 5, 2026, https://academic.oup.com/jcmc/article/25/1/74/5700811
9. Understanding Mechanistic Interpretability in AI Models - IntuitionLabs, accessed February 5, 2026, https://intuitionlabs.ai/articles/mechanistic-interpretability-ai-llms
10. Mechanistic Interpretability: Understanding the Inner Workings of AI Models - Medium, accessed February 5, 2026, https://medium.com/@MakeComputerScienceGreatAgain/mechanistic-interpretability-understanding-the-inner-workings-of-ai-models-399e21d5603c
11. Open Problems in Mechanistic Interpretability - arXiv, accessed February 5, 2026, https://arxiv.org/html/2501.16496v1
12. 7 Types of AI Agent Architecture | Galileo, accessed February 5, 2026, https://galileo.ai/blog/ai-agent-architecture
13. Building AI Agents and Agentic Workflows Specialization - Coursera, accessed February 5, 2026, https://www.coursera.org/specializations/building-ai-agents-and-agentic-workflo

ws
14. 4th International Medical Congress for Students and Young Doctors - Nicolae Testemitanu SUMPh, accessed February 5, 2026, https://repository.usmf.md/bitstream/20.500.12710/20356/1/MedEspera_2012.pdf
15. MODEL AI GOVERNANCE FRAMEWORK FOR AGENTIC AI - IMDA, accessed February 5, 2026, https://www.imda.gov.sg/-/media/imda/files/about/emerging-tech-and-research/artificial-intelligence/mgf-for-agentic-ai.pdf
16. Introducing The Agent Development Lifecycle (ADLC) - Arthur AI, accessed February 5, 2026, https://www.arthur.ai/blog/introducing-adlc
17. The Agent Development Life Cycle - Sierra AI, accessed February 5, 2026, https://sierra.ai/blog/agent-development-life-cycle
18. How to Build a General-Purpose LLM Agent | by Maya Murad | TDS Archive | Medium, accessed February 5, 2026, https://medium.com/data-science/how-to-build-a-general-purpose-llm-agent-c40be49e7400
19. Towards a science of scaling agent systems: When and why agent systems work, accessed February 5, 2026, https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/
20. Agent Skills - Claude API Docs, accessed February 5, 2026, https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
21. Agent Skills: The Universal Standard Transforming How AI Agents Work - Medium, accessed February 5, 2026, https://medium.com/@richardhightower/agent-skills-the-universal-standard-transforming-how-ai-agents-work-fc7397406e2e
22. Heritable epigenetic changes are constrained by the dynamics of regulatory architectures, accessed February 5, 2026, https://elifesciences.org/reviewed-preprints/92093v2